Informe de Actividades — TP N.º 2

Materia: Base de Datos II — Unidad 1: integridad, transacciones y concurrencia Proyecto integrador: Food Store (esquema del TP1) Motor: PostgreSQL 18.6 · Cliente: psql (dos sesiones concurrentes) Base de trabajo: `foodstore_tp2` (copia según protocolo, nunca `foodstore`) Integrantes: Guillot Tiago, Monardez Maximo, Ruiz Matias, Mendez Joaquin, Oviedo Thiago y Chacon Facundo Repositorio: github.com/thiaguchi-stack/DB-TrabajosPracticos-Com4

Regla de fondo: se delega la escritura, nunca la decisión. Este informe recorre las actividades de la consigna, indica el entregable de cada una y resume la evidencia real obtenida en el motor.

Resumen de cumplimiento

Actividad	Entregable	Estado
Parte 0 — Protocolo de seguridad	`protocolo_seguridad.md`	Completa
Parte 1 — Integridad versionada + DUIA	`specs/spec_restricciones.md`, `sql/restricciones_integridad.sql`, `sql/pruebas_restricciones.sql`, `DUIA_parte1.md`	Completa
Parte 2 — Laboratorio de concurrencia + DUIA	`concurrencia/`, `informe/informe_concurrencia.md`, `DUIA_parte2.md`	Completa
Parte 3 — Lectura crítica + DUIA	`informe/ejercicio_lectura_critica.md`, `DUIA_parte3.md`	Completa

Actividad 0 — Protocolo de seguridad

Consigna. Adaptar el protocolo de la cátedra (copia, transacción, respaldo) al entorno real y commitearlo antes de continuar.

Desarrollo. `protocolo_seguridad.md` fija tres pasos obligatorios para toda operación que toque la base:

Paso	Qué exige	Cuándo se salta
1. Copia	Trabajar sobre `foodstore_tp2`, creada desde `foodstore` con `createdb -T` o `pg_dump` + `pg_restore`	Nunca
2. Transacción	Todo script de escritura corre dentro de `BEGIN ... ROLLBACK` y se inspecciona antes del `COMMIT`	Nunca
3. Respaldo	`pg_dump` antes de cualquier cambio estructural (`ALTER`, `DROP`, trigger nuevo)	Nunca

El protocolo incluye un checklist que se repite en cada operación: copia correcta, respaldo tomado, script leído línea por línea, efecto verificado antes del `COMMIT` y, si el script lo generó una IA, `git diff` revisado y DUIA completada.

Verificación. El archivo se commiteó antes de la Parte 1 y se aplicó en todas las pruebas posteriores.

Actividad 1 — Integridad versionada

Consigna. Escribir primero una spec con dos o tres reglas de negocio, generar con IA los triggers que las garantizan en el motor, probarlos con casos válidos e inválidos dentro de una transacción y completar la DUIA.

Spec. `specs/spec_restricciones.md` define tres reglas:

Regla	Enunciado	Tabla donde se valida
R1	Un cliente inactivo no puede generar pedidos nuevos	`pedido` al insertar
R2	Un producto inactivo no se puede vender en líneas nuevas	`linea_pedido` al insertar
R3	La cantidad vendida no puede superar el stock disponible	`linea_pedido` al insertar o actualizar `cantidad`

Queda fuera de alcance descontar stock automáticamente: eso es lógica de negocio transaccional, no una restricción de integridad.

Script. `sql/restricciones_integridad.sql` crea tres funciones y tres triggers `BEFORE INSERT`/`BEFORE UPDATE OF cantidad`, con `RAISE EXCEPTION` y mensajes que incluyen los valores concretos:

Trigger	Regla
`trg_chk_activo_cliente_pedido`	Rechaza pedidos de clientes inactivos o inexistentes
`trg_chk_activo_producto_linea`	Rechaza líneas con productos inactivos o inexistentes
`trg_chk_stock_linea`	Rechaza cantidades mayores al stock del producto

Pruebas. `sql/pruebas_restricciones.sql` aplica el DDL dentro de una transacción y ejecuta bloques `DO` que capturan los errores esperados: cliente inactivo rechazado y activo aceptado; producto inactivo rechazado y activo aceptado; cantidad 5000 sobre stock 20 rechazada y cantidad 3 sobre stock 50 aceptada. Al final inspecciona el estado y hace `ROLLBACK`, de modo que la prueba no persiste cambios.

DUIA. `DUIA_parte1.md` registra el prompt, lo generado, lo aceptado y lo modificado. El cambio principal fue usar `producto_id = 2` en el caso válido de stock para no chocar antes con la PK compuesta `(pedido_id, producto_id)`.

Verificación. Se comprobó la ejecución de los tres triggers con casos válidos e inválidos y que quedaron registrados en `pg_trigger`.

Actividad 2 — Laboratorio de concurrencia

Consigna. Reproducir con dos sesiones concurrentes al menos tres de las cuatro anomalías de la Semana 2, pedir una explicación a la IA, verificarla en el motor real y registrar el resultado. Cada escenario deja su evidencia y su DUIA.

Método. Para cada escenario se ejecutaron dos procesos `psql` en paralelo con `pg_sleep()` como sincronización. La explicación de la IA se contrastó repitiendo el experimento con el nivel de aislamiento propuesto. Vale lo que confirma el motor, no lo que dice el modelo.

Escenario	Fenómeno	Reproducción	Verificación
1	Lectura no repetible	A lee precio en `READ COMMITTED`, B actualiza y confirma, A vuelve a leer	RC: 800 → 850; RR: 800 → 800
2	Lectura fantasma	A cuenta pizzas, B inserta una pizza y confirma, A vuelve a contar	RC: 1 → 2; RR: 1 → 1
3	Espera por bloqueo	A toma `FOR UPDATE` y duerme; B pide `FOR UPDATE` sobre la misma fila	B esperó de 18:05:25 a 18:05:28, tras el `COMMIT` de A
4	Interbloqueo	A bloquea producto 1 y pide 2; B bloquea 2 y pide 1	PostgreSQL abortó A con SQLSTATE 40P01 y B confirmó

Explicación de la IA (síntesis). En `READ COMMITTED` cada sentencia toma un snapshot nuevo, por eso aparecen la lectura no repetible y el fantasma; `REPEATABLE READ` usa un snapshot único por transacción y ambos desaparecen. `FOR UPDATE` toma un bloqueo de fila exclusivo hasta el `COMMIT`/`ROLLBACK`, y la otra sesión queda esperando. El interbloqueo se detecta por el ciclo en el grafo de espera y el motor aborta una víctima con `40P01`; la prevención es acceder siempre en el mismo orden y mantener transacciones cortas.

Evidencia. Las salidas reales están en `concurrencia/escenarioN/salida_*.txt`. Los orquestadores `run1.ps1`, `run1_rr.ps1`, `run2.ps1`, `run3.ps1` y `run4.ps1` usan rutas relativas al script y leen la contraseña de `PGPASSWORD`, de modo que se pueden repetir desde el repositorio. Antes de cada corrida restauran su condición inicial (precio 800, sin `Fugazzeta`, `sync_deadlock.aviso = FALSE`), y `sql/00_seed_minimo.sql` es reejecutable porque trunca y reinicia los IDs; por eso las salidas se pueden regenerar sin limpiar la base a mano. El análisis completo está en `informe/informe_concurrencia.md` y la DUIA en `DUIA_parte2.md`.

Conclusión. La IA acertó en los cuatro escenarios: las explicaciones coincidieron con lo observado en el motor y no hubo discrepancias que documentar.

Actividad 3 — Lectura crítica de scripts de IA

Consigna. Analizar dos scripts generados para dar de baja registros vencidos, indicar qué filas afectaría realmente cada uno, por qué no coincide con la consigna y reescribirlos corregidos.

Script 1 — `UPDATE` sin `WHERE`.

```
UPDATE funcion
SET activa = FALSE;
```

Tal como está, afecta el 100% de la tabla `funcion`: no solo las películas retiradas de cartel, también las que siguen en cartel y las funciones ya pasadas. La consigna pide un subconjunto. La versión corregida agrega un `WHERE` que identifica ese subconjunto:

```
UPDATE funcion
SET activa = FALSE
WHERE pelicula_id IN (
    SELECT id FROM pelicula WHERE en_cartel = FALSE
);
```

En Food Store el equivalente sería `UPDATE producto SET activo = FALSE WHERE categoria_id IN (SELECT id FROM categoria WHERE activo = FALSE);`.

Script 2 — `NOT IN` con `NULL`.

```
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);
```

Si la subconsulta devuelve al menos un `NULL`, la condición `id NOT IN (...)` nunca es verdadera y el `DELETE` afecta 0 filas en silencio; si no hay `NULL`, borra lo esperado. En Food Store la FK es `NOT NULL`, pero en un esquema genérico la columna puede ser nulable. La versión segura usa `NOT EXISTS`:

```
DELETE FROM categoria c
WHERE NOT EXISTS (
    SELECT 1 FROM producto p WHERE p.categoria_id = c.id
);
```

Además, como Food Store usa baja lógica, la acción correcta del dominio sería `UPDATE categoria SET activo = FALSE WHERE NOT EXISTS (...)`.

Conclusión. Los dos casos comparten el patrón de los incidentes reales: sintaxis correcta, intención razonable y un efecto distinto del consignado. El protocolo de la Parte 0 es la defensa: copia, transacción y respaldo, más la lectura humana del script. El análisis completo está en `informe/ejercicio_lectura_critica.md` y la DUIA en `DUIA_parte3.md`.

Actividad 4 — Declaraciones de Uso de IA y verificación humana

Consigna. Completar una DUIA por ejercicio, registrando herramienta, prompt o spec, qué generó, qué se aceptó, qué se modificó y cómo se verificó.

Desarrollo. Las tres DUIA (`DUIA_parte1.md`, `DUIA_parte2.md`, `DUIA_parte3.md`) documentan el uso de OpenCode como motor de escritura y la decisión humana sobre cada script. En los tres casos se registra la misma política: ningún script se ejecuta sin leerse; el DDL y los cambios se prueban primero en una transacción que se revierte; la explicación que vale es la que confirma el motor.

Verificación humana. Cada línea del diff fue leída y entendida antes de aplicarse. Las salidas reales del motor respaldan las afirmaciones del informe de concurrencia. Ninguno de los scripts peligrosos de la Parte 3 se ejecutó contra una base real: se analizaron como ejercicio de lectura crítica.

Checklist final

- [x] `protocolo_seguridad.md` commiteado y aplicado en cada paso.
- [x] Spec escrita antes del script de restricciones.
- [x] Tres triggers probados con casos válidos e inválidos dentro de `BEGIN ... ROLLBACK`.
- [x] Cuatro escenarios de concurrencia reproducidos y verificados con evidencia del motor.
- [x] Dos scripts peligrosos analizados y corregidos.
- [x] Las tres DUIA completas.
- [x] Cada script generado por IA fue leído línea por línea antes de aplicarse.